



East West University
Department of Computer Science and Engineering
A/2, Jahurul Islam Avenue, Aftabnagar, Dhaka.

Topic: Tree, Graph, Traversal Techniques

Course: CSE 207 Data Structures
Instructor: Ahmed Abdal Shafi Rasel

Part 1: Tree

Lab Activities:

- i. Writing tree using structures.
- ii. In order, pre order, post order traversal.

A. Binary Tree using Structure:

```

1 struct Node {
2     int data;
3     struct Node* left;
4     struct Node* right;
5 };
6
7 struct Node* createNode(int value) {
8     struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));
9     newNode->data = value;
10    newNode->left = NULL;
11    newNode->right = NULL;
12    return newNode;
13 }
14

```

```

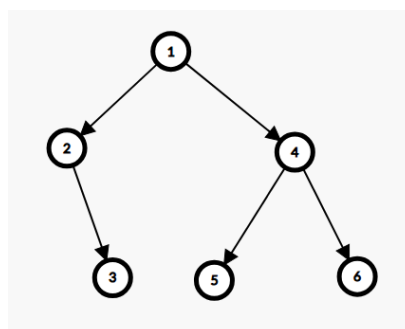
15 struct Node* buildTree() {
16     int value;
17     printf("Enter node value (-1 for no node): ");
18     scanf("%d", &value);
19     if(value == -1) return NULL;
20     struct Node* node = createNode(value);
21     char opt;
22     printf("Does node %d have a left child? (y/n): ", value);
23     scanf(" %c", &opt);
24     if(opt == 'y'){
25         node->left = buildTree();
26     }
27     else{
28         node->left = NULL;
29     }
30     printf("Does node %d have a right child? (y/n): ", value);
31     scanf(" %c", &opt);
32     if(opt == 'y'){
33         node->right = buildTree();
34     }
35     else{
36         node->right = NULL;
37     }
38     return node;
39 }

```

B. Pre-order Traversal (Root, Left Node, Right Node):

<pre> 1 void preorder(struct Node* root) { 2 if(root != NULL) { 3 printf("%d ", root->data); 4 preorder(root->left); 5 preorder(root->right); 6 } 7 } </pre>	<pre> 1 int main() { 2 struct Node* root = buildTree(); 3 printf("Pre Order Traversal:\n"); 4 preorder(root); 5 return 0; 6 } </pre>
---	--

Example:



For this tree, the pre-order traversal will be: 1 2 3 4 5 6

C. Binary Tree using Array:

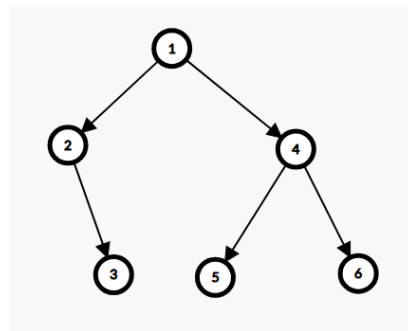
For any node at index i :

- A. Left child $\rightarrow$ index $(2*i + 1)$
- B. Right child $\rightarrow$ index $(2*i + 2)$
- C. Parent $\rightarrow$ index $[(i - 1) / 2]$

Lab Task:

1. Implement the In-order and Post-order traversal for the above tree. (Using list) [Mark: 10]
2. Find the height of a binary tree. [Mark: 10]

Example:



Height = 2

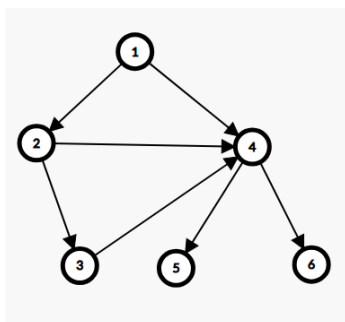
3. Check, if the tree is balanced or not.

Part 2: Graph Representation

Lab Activities:

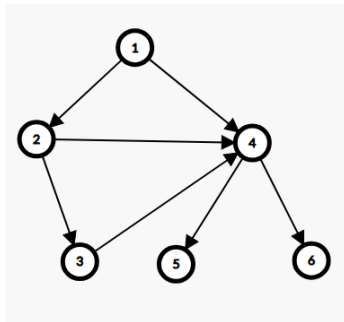
- i. Demonstrating how graphs are represented using adjacency matrix, adjacency list.
- ii. Add edge, remove edge.

a. Adjacency Matrix: (Directed Graph)



	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	1	1	0	0
3	0	0	0	1	0	0
4	0	0	0	0	1	1
5	0	0	0	0	0	0
6	0	0	0	0	0	0

b. Adjacency List:

(Directed Graph)

1	2	4	NULL
2	4	3	NULL
3	4	NULL	
4	5	6	NULL
5	NULL		
6	NULL		

Code:

```

1  const int V = 100;
2
3  struct Node {
4      int dest;
5      struct Node* next;
6  };
7
8  struct Graph {
9      struct Node* adjList[V];
10 };
11
12 struct Node* createNode(int dest) {
13     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
14     newNode->dest = dest;
15     newNode->next = NULL;
16     return newNode;
17 }
18
19 struct Graph* createGraph() {
20     struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));
21     for (int i = 0; i < V; i++)
22         graph->adjList[i] = NULL;
23     return graph;
24 }
25
26 void addEdge(struct Graph* graph, int src, int dest) {
27     struct Node* newNode = createNode(dest);
28     newNode->next = graph->adjList[src];
29     graph->adjList[src] = newNode;
30 }

```

```

1 int main() {
2     struct Graph* graph = createGraph();
3
4     return 0;
5 }

```

Lab Tasks:

1. Implement Adjacency Matrix and Adjacency List, and print the matrix and list. [Marks=10]
(Both for directed, undirected graph.)

Example:

Input:

Add Edge: 1 2

Add Edge: 1 4

Add Edge: 2 4

Add Edge: 2 3

Add Edge: 4 5

Add Edge: 4 6

2. Remove an edge from the graph, and print the matrix and list. (Both for directed, undirected graph.) [Marks=10]

Example:

Input:

Remove Edge: 1 4

Remove Edge: 2 3

3. An adjacency matrix is given, covert it into an adjacency list. [Marks=10]

Example:

	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	1	1	0	0
3	0	0	0	1	0	0
4	0	0	0	0	1	1
5	0	0	0	0	0	0
6	0	0	0	0	0	0

Part 3: Breadth First Search (BFS)

Lab Activities:

- i. Demonstrating how BFS works.

Breadth First Search (BFS) is a fundamental graph traversal algorithm. It begins with a node, then first traverses all its adjacent nodes. Once all adjacent are visited, then their adjacent are traversed.

Code:

```

1  const int V = 100; // Number of vertices
2
3  int visited[V];
4
5  void bfs(struct Graph* graph, int start) { // start = start node
6      for (int i = 0; i < V; i++) visited[i] = 0; // 0 means unvisited
7
8      Queue queue = createQueue(); // Queue implementation (either array or linked list)
9      enqueue(queue, start);
10     visited[start] = 1; // 1 means visited
11
12     while (!isEmpty(queue)) {
13         int current = dequeue(queue);
14         printf("%d ", current); // Printing current node
15
16         struct Node* temp = graph->adjList[current];
17         while (temp != NULL) {
18             int vertex = temp->dest;
19             if (!visited[vertex]) {
20                 enqueue(queue, vertex);
21                 visited[vertex] = 1;
22             }
23             temp = temp->next;
24         }
25     }
26 }

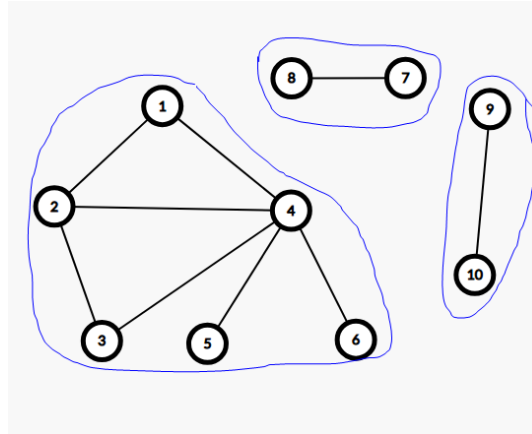
1  int main() {
2      struct Graph* graph = createGraph();
3      // Adding edges to the graph
4      addEdge(graph, 1, 2);
5      addEdge(graph, 2, 3);
6      addEdge(graph, 1, 3);
7      addEdge(graph, 2, 4);
8      addEdge(graph, 2, 4);
9      addEdge(graph, 5, 6);
10
11     // Perform BFS starting from vertex 1
12     bfs(graph, 1);
13     return 0;
14 }

```

Lab Tasks:

1. Print all the connected components of the graph. (Using BFS)
Example:

[Marks=10]



Output:

Component 1:

1 2 4 3 5 6

Component 2:

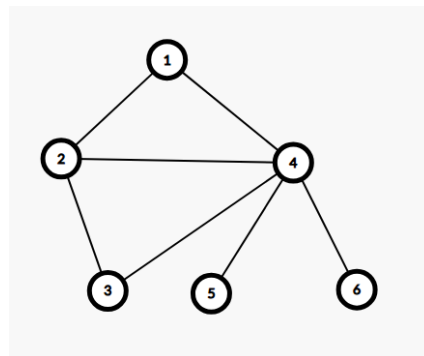
7 8

Component 3:

9 10

2. Find the shortest distance from source to destination node.
Example:

[Marks=10]



Shortest Distance from Node 1 to Node 3?

3. Byteland has **n** cities, and **m** roads between them. The goal is to construct new roads so that there is a route between any two cities.

[Marks=10]

Your task is to find out the minimum number of roads required, and also determine which roads should be built.

The full problem statement is given here: [Link](#)

Part 4: Depth First Search (DFS)

Lab Activities:

- i. Demonstrating how DFS works.

Depth-First Search (DFS) is a graph traversal algorithm that explores as far as possible along each branch before backtracking.

Code:

```

1  const int V = 100; // Number of vertices
2
3  int visited[V];
4
5  void dfs(struct Graph* graph, int v, int visited[]) {
6      visited[v] = 1; // Mark the current node as visited and print it
7      printf("%d ", v);
8
9      struct Node* temp = graph->adjList[v];
10     while (temp != NULL) {
11         if (!visited[temp->dest]) {
12             dfs(graph, temp->dest, visited); // Calling the dfs again to traverse in depth
13         }
14         temp = temp->next;
15     }
16 }

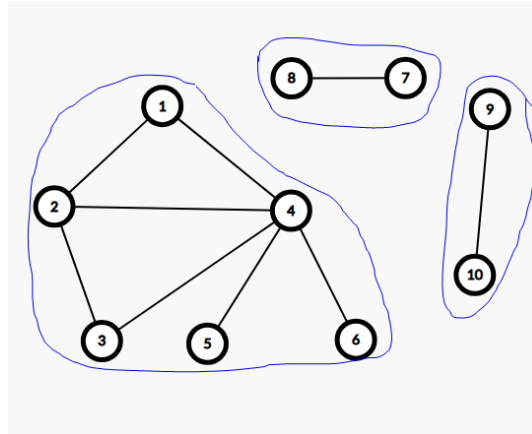
1  int main() {
2      struct Graph* graph = createGraph();
3      // Adding edges to the graph
4      addEdge(graph, 1, 2);
5      addEdge(graph, 2, 3);
6      addEdge(graph, 1, 3);
7      addEdge(graph, 2, 4);
8      addEdge(graph, 2, 4);
9      addEdge(graph, 5, 6);
10
11     // Perform DFS starting from vertex 1
12     dfs(graph, 1);
13     return 0;
14 }

```

Lab Tasks:

1. Print all the connected components of the graph. (Using DFS)
Example:

[Marks=10]



Output:

Component 1:

1 2 4 3 5 6

Component 2:

7 8

Component 3:

9 10

2. Check, if there is any path between source node to destination node. If yes, print "There is a Path!", else print "Impossible!". [Marks=10]

3. Byteland has **n** cities, and **m** roads between them. The goal is to construct new roads so that there is a route between any two cities. [Marks=10]

Your task is to find out the minimum number of roads required, and also determine which roads should be built.

The full problem statement is given here: [Link](#)